

In the command shown in Figure 7, *network create* is used to create user-defined networks. The *--subnet* option is used to specify the subnet in which the containers assigned to this network should reside. Assign *demonet* as the name of the network being created. This command also has a *--driver* option, which takes *bridge* or *overlay*. If the option is not specified, by default it creates the network with the bridge driver. Hence, here the network *demonet* has the bridge network driver.

The containers we create in that network should have IP addresses in that specified subnet. After creating a network with a specific subnet, we can also specify the exact IP address of a container.

```
$docker run --net <user-defined network name> --ip <IP Address> -it ubuntu
```

The command in Figure 8 shows that the *--net* option of the *docker run* command is used to mention the network to which the container should belong. *--ip* is to assign a particular IP address to the container. Do remember that the IP address in Figure 8 should belong to the specified subnet in Figure 7.

Note: We can assign IP addresses to containers only in the user-defined network. We cannot assign specific IP addresses if the container is in the default networks (bridge, host or none). This can be seen in Figure 9.

Creating multiple containers in a specified address range

We have already created a user-defined network *demonet* with a specific subnet. Now, let us learn how to create multiple Docker containers, the IP addresses of which all fall in a given range.

We can write a shell script to create any number of containers, as required. We just have to execute the *docker run* command in a loop. In the *docker run* command, *--net* should

```
➔ ~ docker run --net demonet --ip 173.20.0.10 -it ubuntu
root@39f1fce9e1a1:/#
```

Figure 8: The command to assign the network and IP address to the container

```
➔ ~ docker run --ip 173.20.0.10 -it ubuntu
docker: Error response from daemon: user specified IP address is supported on user defined networks only.
```

Figure 9: Error while assigning the IP address to a container within the default Docker networks

```
echo -n "Please enter number of containers"
read no;

for ((i=1;i<=no;i++))
do
    docker run -itd --net demonet --name container$i ubuntu
done
```

Figure 10: The script to create and run multiple containers in a specified IP address range

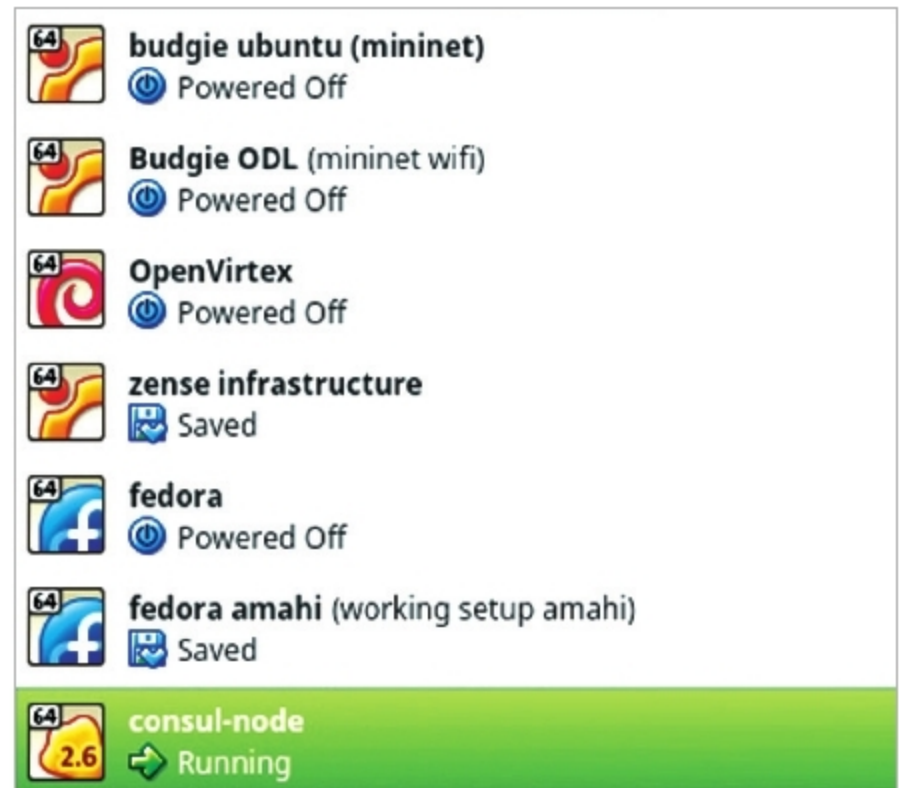


Figure 11: VirtualBox UI with a new virtual host (consul-node) created using Docker Machine

be added to the name of the user-defined network, which in this case is *demonet*. The containers will be created with names *container1*, *container2*, and so on.

The IP addresses of the containers created in Figure 10 will be in a sequence in the given range, specified while creating the user-defined network.

Establishing communication between containers running on different hosts

So far we were dealing with the communication between containers running on a single host, but things get complicated when containers running on different hosts have to communicate with each other. For such communication, overlay networks are used. An overlay network enables communication between various hosts in which the Docker containers are running.

Terminology

Before creating an overlay network, we need to understand a few commonly used terms. These are: Docker Swarm, key-value store, Docker Machine and consul.

Docker Swarm: This is a group or a cluster of Docker hosts that has the Swarm service running in them. This helps in connecting multiple Docker hosts into a cluster using the Swarm service. The hierarchy of the Swarm service contains the master and nodes, which lie in the same Swarm service. First, we create a Swarm master and then add all the other containers as Swarm nodes.

Key-value store: A key-value store is required to store information regarding a particular network. Information such as hosts, IP addresses, etc, is stored in these key-value stores. There are various key-value stores available but, in this article, we will only use the consul key-value store.